

Wallos “Fair Enough” Scheduler

Specification (WFES)

Status: Working Draft

Scope: SMP task scheduling for Wallos

This document specifies the design and behavior of the Wallos “Fair Enough” Scheduler (WFES), a priority-weighted, per-CPU task scheduler for symmetric multiprocessing (SMP) systems.

Contents

Abstract	3
Status of This Document	4
Conventions and Terminology	5
1 Philosophies	5
2 Affinity	6
3 Priority	7
3.1 Fairness	8
3.1.1 Priority 0 and 1	9
3.2 Starvation Mitigation	10
3.2.1 Determining D	10
3.2.2 Hard Cap	11
3.2.3 Examples	11
4 Runqueue Balancing	12
4.1 Load	13
4.2 Moving Tasks	13
4.3 Stealing by an Idle CPU	13
4.4 Periodic Rebalancing	14
4.5 Stealing Tasks With Affinity	14
5 Kernel Tasks	14
5.1 Scheduling Behaviors	15
5.2 Selection and Placement	15
5.3 Preemption	15
5.4 CPU Takeover	16
5.5 Other Preemption	16
6 Yielding	16
7 Blocking	17
8 Blocking Event Handlers	18
8.1 Overview	19
8.2 Registration	19
8.3 Waiting	19
8.4 Event Delivery	19
8.5 Cancellation	20
9 Task Creation	20
9.1 Parameters	21
9.2 Runqueue Assignment	21
9.3 Initial State	21

9.4	Priority and Affinity	21
9.5	Parent/Child Relationships	22
9.6	Spawn Variants	22
9.7	Resource Provisioning	22
9.8	Adopting the Boot Thread	22
10	Task Destruction	22
10.1	Definition	23
10.2	Normal Exit	23
10.3	Cascading Destruction	23
10.4	Destruction of a Blocked Task	23
10.5	External Destruction Requests	24
10.6	Deferred Release	24
11	Timer & Quanta Expiry	24
11.1	Timing Method	25
11.2	Low-Resolution Fallback	25
12	The Scheduling Handler	25
12.1	Order of Operations	26
12.2	The Idle Task	27
13	SMP Synchronization	27
13.1	Rules	28
13.2	Shared State	28
13.3	Cross-CPU Communication	28
13.4	Rebalancing Protocol	29
13.5	Races	29
14	Idle State	29
14.1	Entering Idle	30
14.2	Cooldown Expiry	30
14.2.1	Cooldown Duration Calculation	31
14.3	Receiving Tasks While Idle	31
14.4	Kernel Interaction	31
15	Platform Requirements	32
15.1	Initialization	33
A	Open Design Considerations	33
B	Future Considerations	34
C	Parameters	35
D	Hard Affinity Tasks	36
D.1	Takeover Policy	37
D.2	Telling the Task	37
D.3	Open Questions	37

Abstract

The Wallos “Fair Enough” Scheduler (WFES) is a priority-weighted, per-CPU task scheduler for SMP systems. It is built around simplicity and minimal cross-CPU coordination, favoring predictable, low-overhead behavior over strict fairness guarantees.

Status of This Document

This is a working draft.

It says how the scheduler should behave and which mechanics have to exist. Anything this document does not pin down is implementation defined, and is recorded in the implementation documentation. Items that are intentionally left open are listed in [Appendix A](#). Default values for the scheduler's parameters are collected in [Appendix C](#).

Conventions and Terminology

CPU

A logical compute unit (threads on x86, cores on aarch64, etc.).

BSP

Bootstrap Processor. The CPU that brought up the system, and is typically the “main” CPU from a hardware point of view.

TCB

Task Control Block. The kernel structure holding a task’s saved context, state, and any other per-task bookkeeping.

Kernel Task

These are special tasks created by the kernel, not just a task running in kernel mode (see Section 5)

Normal Task

These are any task that isn’t a kernel task. These can be running in either user or kernel mode.

Owning CPU

The CPU whose `cpu_rq` a task belongs to.

`cpu_rq`

The per-CPU runqueue: one queue per priority level, plus a queue for kernel tasks. It holds every task owned by a given CPU that is not currently executing, including tasks that are blocked (see Section 7).

Quantum

The base unit of scheduling time. The default quantum is 16 ms of runtime.

Quanta

The amount of quantum time allotted to a task at a given priority level.

Eligible

A task is eligible to run when it is not blocked, not suspended, and not marked for destruction.

Context

The saved execution state needed to resume a task.

Idle mask

A global mask with one bit per CPU. A set bit means “this CPU is idle and willing to be handed work”.

Critical Section

A region of code that must not be interrupted or preempted while it executes, typically implemented by disabling interrupts on the current CPU. Only kernel tasks are permitted to enter a critical section (see Section 11).

1 Philosophies

- Each CPU manages its own `cpu_rq` with no global runqueue, allowing each CPU to make scheduling decisions independently. Cross-CPU access to a runqueue is limited to short, locked operations that are intentionally rare (Section 13).
- There is no “main” processor. The BSP holds that role only until the scheduler is initialized and tasks are distributed.
- Priority is weighted. Each priority level receives half the runtime of the one above it.
- Priority 0 and 1 are a special “overriding” class, interleaved between every task transition. These queues are expected to be *mostly* empty under normal operation.
- Fairness is not guaranteed. Starvation is mitigated, but hard latency or throughput guarantees are not made for any given task.
- Affinity binds a task to a specific CPU. Soft affinity is almost always respected, but can be overridden to prevent a CPU from idling indefinitely. Hard affinity is absolute, and tasks using it must tolerate the possibility of never running under certain circumstances.
- Load is distributed lazily. CPUs steal or offer tasks periodically, or immediately when a CPU runs dry. Cross-CPU coordination is intentionally rare.
- Every task transition on a CPU goes through a single scheduling handler (Section 12).
- Blocking is not the scheduler’s responsibility. Subsystems that block tasks own the responsibility of unblocking them. The scheduler only observes whether a task is blocked, and can ask the subsystem to release a task that is being destroyed.
- Only the owning CPU destroys a task. Everyone else requests destruction by marking the task.
- The kernel can preempt anything, at any time, for whatever reason it wants.

2 Affinity

Tasks can be set to only ever run on a single CPU, preventing them from being stolen.

By default, tasks inherit their creator's affinity (Section 9.4). A task with no creator is created **without** affinity.

There are three levels of affinity:

Affinity Level	Description
0 (None)	The task is not set to run on exclusively one CPU.
1 (Soft)	The task is set to run on exclusively one CPU, but can be moved to another if needed.
2 (Hard)	The task is set to run on exclusively one CPU, and cannot be moved to another under any circumstances.

Figure 2.1: Affinity Levels

Setting a soft or hard affinity level without naming a CPU pins the task to its creator's CPU.

Soft affinity is the recommended value for tasks that require affinity. It is incredibly rare for a soft affinity task to be moved to a different CPU. See Section 4.5 for the rules, and Section 5.4 for the one other case in which it moves.

Hard affinity carries the risk that the task may never run if its CPU is taken over by the kernel. See Section 5 for more information.

3 Priority

There are 10 priority levels, 0–9. Priority 0 is the highest; priority 9 is the lowest. The default priority of a newly created task with no creator to inherit from is 5.

Each priority level gets double the runtime of the next lower priority (with the exception of priority 0 and 1), expressed in units of **quantum**, assuming system timer resolution allows it. The amount of time a given task is allotted is called its **quanta**.

The default **quantum** is 16 ms of runtime, summarized in Figure 3.1.

Priority	Time (quanta)	At default quantum
Kernel	1–64, or none	up to 1024 ms
0	8	128 ms
1	4	64 ms
2	8	128 ms
3	4	64 ms
4	2	32 ms
5	1	16 ms
6	1/2	8 ms
7	1/4	4 ms
8	1/8	2 ms
9	1/16	1 ms

Figure 3.1: Priority Quanta

Kernel priority tasks are special tasks created by the kernel and are excluded from all other priority rules. They are discussed in Section 5. In most cases, kernel tasks can be safely ignored from the perspective of an end-user or application programmer. The exception is hard affinity tasks, which may never run again if the kernel permanently takes over their CPU (see Section 5).

3.1 Fairness

With the exception of priority 0 and 1 tasks, all tasks are scheduled round-robin in descending priority order.

For example, the expected scheduling flow, where P# is the priority level:

P2 -> P3 -> P4 -> P5 -> P6 -> P7 -> P8 -> P9 -> P2 -> P3 -> ...

Upon a timeslice expiring or a task yielding, the task is placed at the back of its runqueue. If a given runqueue has no eligible tasks, it is skipped.

For example, if the P4 runqueue is empty, it is simply omitted and the flow proceeds directly from P3 to P5:

P2 -> P3 -> P5 -> P6 -> P7 -> P8 -> P9 -> P2 -> P3 -> ...

Within a runqueue, selection takes the first eligible task, scanning from the front. Tasks that are not eligible (blocked or suspended) are skipped and keep their position, so a task that unblocks is run on its next natural turn, in order.

Any newly added task, regardless of priority, is **not** immediately preemptive. It is added to the back of the runqueue corresponding to its priority level.

3.1.1 Priority 0 and 1

Priority 0 and 1 tasks are considered “overriding.” Their presence means they **must** run if present. Upon the addition of a priority 0 or 1 task, the normal scheduling flow is interrupted and the task is inserted into the next available timeslot.

Priority 0 and 1 tasks are **not** immediately preemptive: they will run after the currently running task’s quanta expires.

Priority 0 and 1 tasks are interleaved between other priority levels:

P0 -> P1 -> P2 -> P0 -> P1 -> P3 -> P0 -> P1 -> P4 -> ...

Between every two normal-priority (P2–P9) runs, the scheduler runs one P0 task if there is an eligible one, then one P1 task if there is an eligible one. An empty P0 or P1 queue is skipped without cost.

It is expected that very few, if any, tasks will ever occupy priority 0 or 1. **Tasks at these levels are expected to use as little of their timeslice as possible.**

3.1.1.1 Intended Use and Quanta Cap

Priority 0 and 1 deviate from the doubling pattern intentionally. Following the pattern strictly would yield 512ms and 256ms respectively. Runtimes this large would dominate the CPU and cause sluggishness in everything else, contradicting their purpose entirely.

P0 and P1 are designed for low-latency work: short bursts that need to run soon, not long computations that need a lot of time. Their quanta are capped at 8 and 4 respectively, matching the P2 and P3 values, to enforce brevity.

Note

If a task genuinely needs more runtime than a P2 priority, it should likely be implemented as a kernel task (see Section 5).

Because P0 and P1 are interleaved between every task transition, their effective CPU share is disproportionately high relative to their raw quanta. Even at modest quanta values, a consistently occupied P0 or P1 runqueue can dominate scheduling. The quanta cap is therefore a necessary constraint to keep the system functional under any load.

A well-behaved P0 or P1 task should yield well before its quanta expires. Consistently consuming the full allotment is a strong signal that the task is misusing its priority level. The scheduler does not enforce this, but it is a design expectation.

The scheduler makes no distinction between a task that uses 1ms of its quanta and one that uses the full allotment. Both are treated identically at the scheduling level. The responsibility for brevity lies entirely with the task.

3.2 Starvation Mitigation

To prevent long runqueues from starving other tasks, the scheduler may run multiple tasks from the same priority level back-to-back. This applies only to P2-P9. P0 and P1 are unaffected, as their interleaving is unconditional.

Starvation mitigation does not alter priority ordering. It only affects how many times a given priority level is selected before descending to the next.

The number of consecutive runs granted to a given priority level is given in Formula 1. It is computed once, at the moment the level is chosen, and the extra runs (the *burst*) are then served from that level before the rotation moves on. P0 and P1 tasks still interleave between the individual runs of a burst. If the level runs out of eligible tasks partway through, the burst simply ends.

Formula 1: Back-to-Back Run Count

$$\text{back_to_back} = \text{clamp}(\text{floor}(\text{len}(\text{rq}) / D), 1, 8)$$

Where:

- $\text{len}(\text{rq})$ is the number of tasks in that priority's runqueue, including any that are currently blocked, and including the task that was just selected.
- D is a dynamic divisor determined by overall system load (see Section 3.2.1).
- $\text{clamp}(x, 1, 8)$ ensures a minimum of 1 (normal behavior) and a hard maximum of 8 consecutive runs.

A result of 1 is the default behavior (no extra consecutive runs). Starvation mitigation only meaningfully activates when the result exceeds 1.

Note

$\text{len}(\text{rq})$ counts every task assigned to the runqueue, not only the ones currently eligible to run. A runqueue with many blocked tasks and few runnable ones is still treated as a long runqueue for this formula.

3.2.1 Determining D

Only **non-empty** runqueues (P2-P9) are considered when computing D . Runqueues with 0 tasks are excluded entirely, as they have no bearing on scheduling pressure. Runqueues with fewer than 10 tasks are still included: they are short, but they still must be considered for contention.

A runqueue with 20 or more tasks is called *long*. D is determined by how many of the non-empty runqueues are long:

Formula 2: D (Concentrated Load, ≤ 3 runqueues with 20+ tasks)

$$D = \max(\text{longest_short_rq}, 10)$$

Formula 3: D (Distributed Load, > 3 runqueues with 20+ tasks)

$$D = \max(\text{floor}(\text{avg_rq_len}), 10)$$

Where:

- `longest_short_rq` is the length of the longest non-empty P2-P9 runqueue that is **not** long (fewer than 20 tasks), or 0 if there is none.
 - The long queues are deliberately excluded. Using the longest long queue as the divisor would push the quotient to about 1 for exactly the queue that needs relief.
- `avg_rq_len` is the average length across all **non-empty** P2-P9 runqueues.
- The `max(..., 10)` term acts as a floor in both cases, treating 10 as the baseline normal runqueue length and preventing disproportionate results when all queues are short.

3.2.2 Hard Cap

`back_to_back` is capped at 8.

Without a cap, an extremely large runqueue could still produce an unreasonably large consecutive run count despite the dampening effect of `D`. The cap ensures that no single priority level can monopolize the CPU under extreme load.

The value 8 was chosen to:

- Provide meaningful starvation relief under heavy imbalance.
- Preserve responsiveness for other priority levels.

3.2.3 Examples

Note

For brevity, the examples in this section assume P0 and P1 have zero tasks, and are therefore omitted from the flow diagrams below. A separate example showing P0/P1 interleaving follows at the end of this section.

3.2.3.1 Few Long Runqueues

(≤ 3 non-empty queues with 20+ tasks: use Formula 2.)

Given:

P2=4, P3=2, P4=26, P5=34

Only P4 and P5 are long, so the longest short queue is P2 with 4 tasks and $D = \max(4, 10) = 10$:

P4: `clamp(floor(26/10), 1, 8) = 2`

P5: `clamp(floor(34/10), 1, 8) = 3`

Resulting flow:

P2 -> P3 -> P4 -> P4 -> P5 -> P5 -> P5 -> P6 -> ...

3.2.3.2 Many Long Runqueues

(>3 non-empty queues with 20+ tasks: use Formula 3.)

Given:

P2=25, P3=30, P4=26, P5=34, P6=2, P7=2

P8 and P9 are empty and excluded. Four queues are long, so:

```
avg = floor((25+30+26+34+2+2) / 6)
    = floor(119/6)
    = 19
```

```
D = 19
```

```
P2: clamp(floor(25/19), 1, 8) = 1
P3: clamp(floor(30/19), 1, 8) = 1
P4: clamp(floor(26/19), 1, 8) = 1
P5: clamp(floor(34/19), 1, 8) = 1
P6: clamp(floor(2/19), 1, 8) = 1
P7: clamp(floor(2/19), 1, 8) = 1
```

When load is broadly distributed, the average rises and D rises with it, dampening back-to-back counts across the board. Starvation mitigation is most aggressive when load is concentrated in one or two runqueues, and least aggressive when all runqueues are similarly loaded. This is the desired behavior.

Even under extreme imbalance (e.g., thousands of tasks in one runqueue), the clamp guarantees that no priority level will run more than 8 consecutive times before descending to the next.

3.2.3.3 With a P0 or P1 Task Present

Using the same P2=4, P3=2, P4=26, P5=34 scenario above, a single P1 task is interleaved between every transition:

```
P2 -> P1 -> P3 -> P1 -> P4 -> P1 -> P4 -> P1 ->
P5 -> P1 -> P5 -> P1 -> P5 -> P1 -> P6 -> ...
```

4 Runqueue Balancing

Other CPUs will occasionally steal tasks from other CPUs if their own runqueue is sufficiently empty. Similarly, a CPU may offer some of its tasks to another CPU if there is a significant imbalance in runqueue sizes.

Rebalancing is only triggered periodically, on the order of every few seconds.

If a CPU completely runs out of runnable tasks, it will immediately attempt to steal several tasks from other CPUs. A cooldown period follows a failed attempt, during which the CPU will not attempt to steal again (see Section 14.2.1).

4.1 Load

Balancing decisions use a cheap, approximate measure of a CPU's load: the number of user tasks it owns (including blocked ones), plus the one it is currently running. Kernel tasks are not counted. Priorities are not weighted. Other CPUs read this measure without synchronizing with its owner, so it may be slightly stale. Balancing is designed to tolerate that.

4.2 Moving Tasks

Stealing, offering, and takeover redistribution all move tasks under the same rules.

A task may move only if all of the following hold:

- It is not a kernel task.
- It is not marked for destruction. Such a task is left for its owner to destroy.
- It is eligible to run. The only exception is takeover redistribution, which also moves blocked and suspended tasks, because they would otherwise wake on a CPU that no longer runs tasks.
- Its affinity level allows it (Section 4.5). Hard affinity tasks never move.

Candidates are taken from the **back** of the source `cpu_rq`, scanning the priority levels P2 through P9 in that order.

P0 and P1 are never moved as they are latency-critical and expected to be nearly empty.

The synchronization required while moving tasks is given in Section 13. If the destination is idle, it must be woken so that it notices the new work.

4.3 Stealing by an Idle CPU

When a CPU has nothing to run and is not cooling down, it steals up to a fixed number of tasks (the *steal batch*). It visits the other CPUs round-robin, starting with the CPU after itself, so thieves do not all go to the same victim.

From each victim with queued tasks it takes about half of the victim's load (div/2, rounded down).

It never takes more than what remains of the batch.

4.4 Periodic Rebalancing

Every few seconds (the *rebalance interval*) each CPU runs a balance pass from its own scheduling handler. Passes on different CPUs are not synchronized.

1. Find the busiest and the least busy CPU among those that are online and not taken over.
2. If they are the same CPU, or their loads differ by less than the *minimum imbalance*, do nothing. This prevents tasks from bouncing over small differences.
3. Otherwise the amount to move is half the difference, capped at 8 tasks per pass to bound the time spent with locks held and interrupts off.
4. If the running CPU is the busiest one, it **offers** that many tasks to the least busy CPU.
5. Otherwise, if its own load is at most half of the busiest CPU's load, it **steals** half the difference between the busiest CPU's load and its own, again capped at 8.
6. Otherwise it does nothing. The busiest or the emptiest CPU will handle the imbalance on its own pass.

Periodic balancing never moves soft affinity tasks or blocked tasks. A blocked task adds no runnable load, so moving it would not balance anything.

4.5 Stealing Tasks With Affinity

Soft affinity tasks may only be stolen by an idle CPU, and only when **all** of the following conditions are true:

- The stealing CPU has no other tasks of any priority (its load is 0).
- All other CPUs, including the one owning the target task, have *only* tasks with affinity: none has a queued task without affinity, and none is currently running one.
- The target task is **not** the only task on its owning CPU.

Due to the specificity of these conditions, soft affinity tasks are very rarely stolen, and only to prevent a CPU from idling indefinitely.

Soft affinity tasks also move when their CPU is taken over by the kernel (Section 5.4).

Tasks with hard affinity can never be stolen.

Note

There will likely be a way to move hard affinity tasks, only in the event of a takeover, in future spec revisions.

5 Kernel Tasks

The kernel reserves the right to immediately preempt any task of any priority.

A *kernel task* is a special task. It runs exclusively in kernel mode. This **does not** mean all kernel mode tasks are *kernel tasks*. A *normal task* can be run from a kernel context, with normal task behaviors.

Kernel tasks are queued separately from the priority levels and do not have to respect any priority rules. They are created through the same mechanism as any other task (Section 9), and are never moved by balancing.

5.1 Scheduling Behaviors

When a kernel task is created, its creator selects how it is to be scheduled (Figure 5.1).

Behavior	Description
Next timeslice	The default. Runs in the next timeslice, once the current task's quanta expires or it yields.
Immediate preemption	Preempts the running task right away (Section 5.3).
Interleaved	Runs between regular task transitions (like priority 0 and 1), at most once between two normal tasks, so that it cannot run back to back.
Takeover	Forcefully takes over the CPU (Section 5.4).
Run to completion	Is not time sliced: no quanta limit is enforced while it runs.

Figure 5.1: Kernel Task Scheduling Behaviors

Where it is meaningful, behaviors may be combined. A kernel task that is time sliced requests a number of quanta from 1 to 64 (1024 ms at default timing). Larger requests are clamped to 64.

5.2 Selection and Placement

At every scheduling decision, kernel tasks are considered before anything else: the first eligible kernel task is selected, subject to the interleaving rule above. If none is selected, the CPU's displaced task (if any) resumes, and only then does the normal P0/P1/P2–P9 flow run.

New kernel tasks are placed like any other task (Section 9.2): on the CPU named by an affinity target, otherwise on an idle CPU if one is advertising, otherwise on the creator's CPU. An idle CPU is therefore the preferred target. If no CPU is idle, the kernel falls back to its normal preemption behavior.

5.3 Preemption

A kernel task with immediate preemption (or takeover) interrupts its CPU so that the CPU ends the running task's quanta at once.

When the CPU is running a user task, the preempted task is **not** moved to the end of the runqueue, and the runqueue position is **not** updated. Instead it is held aside, together with whatever remained of its quanta:

P2 -> P3 (preempted by kernel) -> PK -> Same P3 task -> P4 -> ...

The displaced task resumes as soon as no kernel task is eligible, and runs only for its remaining quanta.

Only one task can be displaced at a time. If a task is preempted while another is already displaced, or if the preempted task is itself a kernel task, it is requeued at the back of its queue normally. If the displaced task blocks or is suspended while waiting, it returns to the front of its priority's queue, keeping its place in line. If the CPU is idle when the preemption arrives, there is nothing to displace and the kernel task simply runs.

A perpetually runnable kernel task will starve everything else on its CPU, including a displaced task. This is by design (the kernel can do whatever it wants), but see [Appendix A](#).

5.4 CPU Takeover

A kernel task created with the takeover behavior takes its CPU over. A CPU stays taken over until the last takeover task on it has been destroyed.

While a CPU is taken over:

- Only kernel tasks are served. User tasks are never selected.
- On every scheduling pass, all user tasks leave the CPU, **excluding** tasks with hard affinity. This includes queued, blocked, suspended, and soft affinity tasks, and the displaced task (which is first returned to a queue). They are split evenly, round-robin, among the other CPUs that are not themselves taken over. If a destination cannot be used at that moment the attempt is simply retried on the next pass.
- The CPU does not steal tasks, and does not take part in periodic balancing.

Nothing moves back when the takeover ends: a moved task stays on its new owning CPU.

Hard affinity tasks must be designed to tolerate the possibility of never running if the kernel takes over their CPU indefinitely. The kernel will provide some form of signal to such tasks when a takeover begins, and the task will be given a *very short* quanta to handle this notice.

Note

The signaling mechanism is yet to be determined, see [Appendix A](#).

5.5 Other Preemption

The kernel is the only entity in the system permitted to end a task's quanta before it expires, other than the task itself yielding or exiting. An externally requested kill ([Section 10.5](#)) is never applied mid-quanta, regardless of its source.

There are no other cases where a task may be preempted by the addition of another task. All other preemptions occur only once the current task's quanta has expired.

6 Yielding

Yielding keeps the task in its `cpu_rq` but requests that the CPU switch context to the next task before the yielding task's normal timeslice expires. This is intended to replace busy-waiting in non-blocking functions.

A yielding task goes to the back of its priority's queue, any unused quanta are discarded, and the next task's quanta begins.

7 Blocking

A task never leaves its `cpu_rq` while blocked. Every task carries an atomic **blocked** flag in its TCB, set and cleared only by the subsystem it is waiting on.

The scheduler skips any task with its blocked flag set during a normal scheduling pass, without removing it from its `cpu_rq`. The scheduler has no knowledge of what a task is blocked on, or which subsystem holds it. It only observes the flag, and relies on the subsystem's cancellation entry point when a blocked task is destroyed (Section 8.5).

A task that is woken is not run immediately. It runs on its next natural turn, in order. If its CPU is idle, the CPU is woken so that it notices.

The responsibilities a blocking subsystem must satisfy, including how it registers, waits, wakes, and cancels tasks, are defined in Section 8.

8 Blocking Event Handlers

A blocking event handler is any subsystem that can suspend a task pending some condition it alone is responsible for tracking (a syscall implementation, a driver, a timer, etc.). This section defines the contract such a subsystem must satisfy. It applies equally to kernel subsystems and to any other component that blocks tasks on its own conditions.

8.1 Overview

A blocking subsystem maintains its own list of tasks waiting on it. WFES does not manage this list, and has no visibility into what a task is waiting for (Section 7). The subsystem alone is responsible for adding tasks to this list, delivering results, and removing tasks from it.

8.2 Registration

When a task blocks on a subsystem, the subsystem is responsible for:

1. Setting the task's blocked flag, and making the subsystem's cancellation entry point (Section 8.5) reachable from the task. How that association is stored is implementation defined. It may be omitted if nothing holds a reference to the task.
2. Recording a reference to the task in its own waitlist.

Both steps must happen while holding the same lock the subsystem's delivery path takes, and the task must wait only afterwards (Section 8.3).

Note

Lost wakeups. A delivery that clears the blocked flag *before* it was set does nothing, and the task would then block forever. Setting the flag under the subsystem's lock, before the waitlist entry becomes visible to the delivery path, makes that impossible. A combined register-and-wait operation, if an implementation offers one, is only safe when the delivery cannot run before it is made (for example, it is triggered by something started afterwards, or by a periodic timer that will simply fire again).

8.3 Waiting

After registering and releasing its lock, the task gives up the CPU until its blocked flag clears. The scheduler skips the task meanwhile, so blocking costs nothing but the task's queue slot. If the task is destroyed while waiting, the wait never returns.

Waiting must happen in task context, never from an interrupt handler.

8.4 Event Delivery

When the condition a subsystem is waiting on is satisfied, the subsystem is responsible for:

1. Delivering the relevant result to each waiting task.
2. Removing the task's reference from its own waitlist.
3. Waking the task, which clears its blocked flag.

Waking reports whether it succeeded. It fails if the task is being destroyed, in which case the delivery must be discarded. Waking an unblocked task is a harmless no-op.

The subsystem must not touch the task after waking it: once the flag is cleared the task may run and be destroyed immediately.

Waking by task identifier (Section 9) is also required, for contexts that cannot hold a task reference safely, such as timer callbacks. It fails if the task no longer exists. Both forms must be safe to use from interrupt context.

8.5 Cancellation

Every blocking subsystem that holds a reference to a task must provide a cancel entry point. This entry point is invoked by task destruction (Section 10.4) for any task that was blocked at the time of destruction.

A subsystem's cancel entry point must, at minimum, remove its reference to the cancelled task from its own waitlist before returning, and must be safe to call when the reference is already gone. This guarantees the subsystem holds no reference to the task once destruction completes, regardless of whether the subsystem's awaited condition ever occurs.

Event delivery and cancellation can occur concurrently, on different CPUs. A subsystem must never act on a delivery for a task that has already been cancelled, even if the two happen at nearly the same time. Waking reports failure for a task already marked as dying, which gives the delivery path the signal it needs.

Note

The bookkeeping used to satisfy this is left to the subsystem, since it depends heavily on how that subsystem tracks its own waiting tasks. One common approach is for the cancel entry point to record the cancelled task's reference somewhere the delivery path checks before acting on it, so a late or in-flight delivery is silently discarded rather than acting on a task that no longer exists. Whatever the mechanism, the outcome is mandatory: a cancelled task is never acted upon again.

9 Task Creation

A task is created by a currently running task, kernel or user, through the kernel's task-creation call. The task requesting creation is referred to below as the **creator**. During early boot there may be no creator; defaults are used in that case.

Every task has a unique identifier that is never reused. Identifiers are how tasks are referred to from outside the scheduler (killing, resuming, waking) without keeping a reference that could dangle.

9.1 Parameters

Creation takes: a name, an entry function and its argument, optionally a priority, an affinity level and target CPU, an initial state, how the task is related to its creator (Section 9.6), for kernel tasks the scheduling behavior and quanta (Section 5), and optionally a stack size. Anything not given is inherited or defaulted as described below.

9.2 Runqueue Assignment

Assignment does not consult the load of other CPUs.

1. If an affinity target applies (explicit, or inherited per Section 9.4), the new task is placed on that CPU.
2. Otherwise, if the idle mask (Section 14.3) indicates an idle CPU, one is claimed atomically and the new task is placed directly on that CPU's `cpu_rq`.
3. Otherwise, the new task is placed on the back of the creator's own `cpu_rq`.

If the destination is another CPU and that CPU is idle, it is woken so that it notices the new task. The new task is never immediately preemptive.

9.3 Initial State

A newly created task defaults to **ready**, and is eligible for scheduling as soon as it is placed on a `cpu_rq`.

The creation call may request a different initial state, such as **suspended**, so that a debugger or supervising task can inspect or modify the new task before it runs for the first time. A task created in a non-ready state is still assigned a `cpu_rq` per Section 9.2, but is not scheduled until explicitly resumed. A task can also be suspended later, by identifier. A suspended task is not eligible, and this takes effect at the owning CPU's next scheduling decision.

9.4 Priority and Affinity

By default, a newly created task inherits both its priority (Section 3) and its affinity level (Section 2) from its creator. If the creator is a kernel task, or there is no creator, the defaults are priority 5 and no affinity.

If the creator has soft or hard affinity to a given CPU, an inheriting child receives the same affinity level to the same CPU, and is placed directly on that CPU rather than through Section 9.2. Note that this means the children of a pinned task are pinned too, unless they override it.

The creation call may override either value explicitly. An override always takes precedence over inheritance. Kernel tasks always have the kernel priority and no affinity.

9.5 Parent/Child Relationships

Every task records a single parent, established at creation and fixed for the task's lifetime, unless the parent is destroyed first (Section 10.3).

Wait-for-exit, exit status, and signal delivery between parent and child are not currently planned, but may be added in the future. See Appendix B.

A task created with no parent (see Section 9.6) is not subject to cascading destruction.

9.6 Spawn Variants

The default creation call assigns priority, affinity, and parentage entirely by inheritance from the creator, as described above. The following variants are also available, and may be combined:

- **Explicit priority.** The new task is assigned a specific priority level rather than inheriting the creator's.
- **Explicit affinity.** The new task is assigned a specific affinity level and, where applicable, a specific target CPU, rather than inheriting the creator's.
- **Sibling spawn.** The new task's parent is set to the creator's own parent, rather than to the creator itself. Destruction of the creator does not destroy a task spawned this way.
- **Detached spawn.** The new task is created with no parent at all, and is exempt from cascading destruction entirely.

9.7 Resource Provisioning

The kernel is responsible for providing the backing storage a new task requires, at minimum a stack and a TCB. A task's resources must not depend on those of another task, unless that dependency is explicit and intentional, such as a deliberately shared memory region. The allocation strategy the kernel uses to provide this storage is outside the scope of this specification.

A new task's initial context is prepared so that its first resumption calls its entry function with its argument. Returning from the entry function is a normal exit (Section 10.2).

9.8 Adopting the Boot Thread

The thread that ran the boot code on the BSP is not created through the creation call. Before preemption starts on the BSP it is *adopted* as a task, with the default priority and soft affinity to the BSP. Its stack was not allocated by the scheduler, so the scheduler never frees it. The thread then keeps running, now as an ordinary task.

10 Task Destruction

10.1 Definition

Destroying a task means, at minimum:

- Marking it dying, so that any wake that races with destruction fails (Section 8).
- If it was blocked, invoking the blocking subsystem's cancel entry point so it drops its reference (Section 10.4).
- Making it unreachable from outside: it can no longer be found by identifier, and it is removed from its parent's children.
- Applying cascading destruction to any of its children (Section 10.3).
- If it held a CPU takeover, ending that takeover.
- Returning its stack, TCB, and any other per-task resources to the kernel (Section 10.6).

Destruction is always carried out by the owning CPU, while the task is not running and is not in any runqueue, and with no runqueue lock held. Before that point, it is only marked for destruction (see Section 10.4 and Section 10.5).

10.2 Normal Exit

A task is destroyed normally when it returns from its entry point, or explicitly calls an exit routine. Both cases are treated identically: the task marks itself for destruction and yields, and the yield never returns to the task. The owning CPU carries out destruction (Section 10.1) in that same scheduling pass.

Because destruction is always initiated and completed by the CPU that already owns the task's `cpu_rq`, no cross-CPU notification is required for the normal exit path.

Zombie and wait-for-exit-status semantics are not currently defined, but may be introduced in the future. See Appendix B.

10.3 Cascading Destruction

Destroying a task also destroys every task that records it as a parent (Section 9.5), recursively. Each child is orphaned, marked for destruction, and its owning CPU is woken if that is a different CPU. The children are destroyed by their own owning CPUs, which in turn cascade to their children. Sibling-spawned and detached tasks (Section 9.6) are unaffected by the destruction of the task that created them.

10.4 Destruction of a Blocked Task

A blocking subsystem cannot be made to wait for its own event before releasing a task, since that event may never occur. Destroying a blocked task therefore requires the owning subsystem to release its reference to the task immediately, rather than destruction waiting on the subsystem.

Destruction achieves this by atomically marking the task dying and then, if the task was blocked at that moment, invoking the subsystem's cancel entry point (Section 8.5). A task that is not blocked needs no cancellation.

10.5 External Destruction Requests

A destruction request originating from a task other than the target (e.g. a `pkill`-style call) names the target by identifier. It cannot directly remove the target from its `cpu_rq`, since only the owning CPU removes tasks for destruction (Section 13).

1. The requester looks the target up by identifier. If it no longer exists, the request fails.
2. The requester atomically marks the target as pending-kill.
3. If the requester is not running on the target's owning CPU, it sends that CPU a reschedule IPI. An idle CPU acts on it immediately. A busy CPU ignores it.
4. The owning CPU destroys the task following the normal path (Section 10.2) at a scheduling decision, in one of two ways:
 - if the task is the one running, when its quanta ends, as the outgoing task;
 - otherwise, when selection scans the runqueue the task is in. A task marked for destruction is removed and destroyed whether or not it is blocked.

An externally requested kill is never applied mid-quanta. Only the kernel may end a task's quanta early (Section 5).

The scheduler gives no bound on how long a kill takes beyond “at the owner's next scheduling decision that scans that queue” (see Appendix A).

10.6 Deferred Release

Destruction normally runs inside the scheduling handler, which may be executing on the dying task's own stack (it was the outgoing task). The stack and TCB must therefore not be freed immediately. Their release is deferred until a later invocation of the scheduling handler on the same CPU, which is always on a different stack.

11 Timer & Quanta Expiry

11.1 Timing Method

Each CPU is responsible for its own quanta expiry using an independent, per-CPU one-shot timer. After every scheduling decision the timer is re-armed to fire at the next task's quanta boundary, and disarmed if the next task has no quanta limit (a kernel task that runs to completion).

The scheduler requires one-shot behavior from the platform (Section 15). A platform whose hardware is only periodic must emulate it, for example by counting ticks until the current task's quanta has elapsed.

Time is measured in microseconds from a monotonic, per-CPU clock. Timestamps (slice start, steal cooldown, next rebalance) are only ever compared with later readings of the *same* CPU's clock.

No kernel or user code may assume that any particular timing scheme is present. A regular task must not assume it will run uninterrupted for any period of time, including the duration of its own quanta. Kernel tasks are the exception: they may enter a critical section, disabling or enabling interrupts as needed.

11.2 Low-Resolution Fallback

If a CPU's timer cannot achieve the granularity required by a given priority level, that level's quanta is clamped to the timer's minimum achievable interval.

12 The Scheduling Handler

Each CPU runs a single scheduling handler, responsible for every transition between tasks. It is entered with interrupts disabled, with the interrupted context saved, for one of the reasons in Figure 12.1. The handler decides which task runs next and resumes it. If it decides that the interrupted task keeps running, that task simply resumes.

Reason	Meaning
Timer	Quanta expiry, or an idle CPU's periodic wake.
Yield	The running task yielded or exited (Sections 6 and 10.2).
Reschedule IPI	Another CPU wants this CPU to look for work. Acted on only if this CPU is idle, otherwise it's ignored.
Kernel preempt	The kernel is ending the running task's quanta early (Section 5.3). Ignored if nothing is running yet.

Figure 12.1: Handler Entry Reasons

A timer interrupt that arrives before the CPU has entered its scheduling loop is ignored.

12.1 Order of Operations

On each invocation that is acted upon, the handler performs, in order:

1. Free any tasks destroyed by earlier invocations on this CPU (Section 10.6).
2. If a task was running, save its context.
3. Dispose of the outgoing task (the idle task is never queued):
 - (a) If it is marked for destruction, queue it for destruction instead of requeuing it.
 - (b) Otherwise, if this is a kernel preemption of a user task and no task is displaced yet, hold it aside with its remaining quanta (Section 5.3).
 - (c) Otherwise, if it yielded or its quanta expired, place it at the back of its priority's run-queue.
4. If the CPU is taken over, return any displaced task to a queue and redistribute the user tasks (Section 5.4).
5. If this CPU's rebalance time has come, run a periodic balance pass (Section 4).
6. Select the next task to run: kernel tasks, then the displaced task, then priority ordering (Section 3), the priority 0/1 interleave, and starvation mitigation (Section 3.2). Tasks that are not eligible are skipped, and tasks marked for destruction found during the scan are removed and destroyed (with no runqueue lock held).
7. If no eligible task is found, and the CPU is not taken over, attempt to steal once (Section 14) and select again if tasks arrived.
8. If still none, select the CPU's idle task (Section 12.2).

9. Re-arm the timer, where applicable, for the selected task's quanta. A task resuming from being displaced gets only its remaining quanta. The idle task gets the idle wake interval.
10. If the selected task is not the idle task, withdraw the CPU from the idle mask.
11. Restore context and resume execution.

Kernel preemption (Section 5) may enter this same handler outside of its normal invocation points, at the kernel's discretion.

Steps that touch another CPU's runqueue (redistribution, balancing, stealing) run with this CPU's own runqueue lock released.

12.2 The Idle Task

Each CPU has exactly one idle task. It is not created through the normal task-creation path (Section 9), is never placed in `cpu_rq`, and cannot be destroyed, stolen, or assigned affinity. It needs only a small stack, since it does nothing but halt.

The idle task is selected only when the scheduling handler finds no eligible task on the CPU. All idle logic (cooldown, stealing, donation) lives in the handler, not in the idle task, which only halts the CPU until the next interrupt. See Section 14.

The idle task is preempted the moment any other task becomes eligible to run on that CPU and another CPU wakes it, whether through creation, an offered task, a successful steal, or a blocked task's flag clearing. It does not wait for a quanta boundary, since it holds no quanta of its own. Should a wake-up be missed because it raced with the CPU going idle, the idle wake interval bounds the delay (Section 14).

13 SMP Synchronization

13.1 Rules

- Each runqueue is protected by its own lock. The owning CPU takes it on every scheduling pass.
- **A CPU must never hold more than one runqueue lock at a time.** This removes any possibility of a deadlock between runqueues, and the need for a lock ordering.
- Runqueue locks are held with interrupts disabled on the local CPU. The scheduling handler takes them from interrupt context, so a holder interrupted by the handler would otherwise deadlock on its own lock.
- When moving tasks, the source runqueue lock is only *tried*. If it is busy, that CPU is skipped.
- No runqueue lock is held while destroying a task, because cancel entry points belong to other subsystems and may take locks of their own. The same goes for calling any other code outside the scheduler.
- Whatever can be a single atomically accessed value rather than something under a lock, should be.
- Only the owning CPU removes a task for destruction. Other CPUs mark the task, and the owner acts on the mark.

13.2 Shared State

- **Runqueues** are protected by their locks. They are modified by the owner on each pass, and by other CPUs only when inserting a new task or moving tasks.
- **State other CPUs read without taking a lock** is accessed atomically, and may be stale. This is the load measure used for balancing, the owning CPU of a task, and the markers on a task (blocked, suspended, pending-kill, dying).
- **The idle mask** is manipulated with atomic bit operations. A CPU is claimed by atomically clearing its bit, so that exactly one claimant wins.
- **Whatever lets tasks be found from outside their owner** (lookup by identifier, the parent/child relationships) needs its own synchronization. That synchronization is never held together with a runqueue lock.
- **Everything else** (what a CPU is running, its selection state, its timing state) is touched only by the owning CPU.

13.3 Cross-CPU Communication

CPUs communicate in only three ways:

- **Locked runqueue operations:** inserting a task into another CPU's queue (creation, moving) or removing tasks from it (moving). See the protocol below.
- **Markers on tasks:** pending-kill, suspended, blocked.

- **Inter-processor interrupts:** a *reschedule IPI* wakes an idle CPU. A *kernel-preemption IPI* makes a CPU end its running task's quanta. Neither carries data. A busy CPU ignores a reschedule IPI and finds the work at its next scheduling decision.

13.4 Rebalancing Protocol

Moving tasks from one CPU to another proceeds as follows:

1. Try to lock the source runqueue. If it is busy, give up and skip that CPU.
2. Detach the chosen tasks into a private chain, preserving their relative order.
3. Unlock the source.
4. Lock the destination, add each task to the back of its priority's queue (which makes the destination the task's owning CPU), and unlock.
5. If the destination is idle, wake it.

Between detaching and attaching, a task is in no runqueue. It remains reachable by identifier, so it can still be killed, suspended, resumed, or woken during that window.

13.5 Races

13.5.0.1 Stealing versus destruction.

A task is destroyed only by its owner, and only after it has been removed from the queue under the queue lock. Stealers skip tasks that are marked for destruction, so a task being killed is never moved. A task in transit between queues is not reachable by any scan, so it cannot be destroyed in that window. If it was killed meanwhile, its new owner finds the mark.

13.5.0.2 Wake versus destruction.

Destruction atomically marks the task dying and calls the cancel entry point only if the task was blocked at that instant. A wake clears the blocked flag only if the dying mark is clear, and reads everything it needs from the task before doing so. Exactly one of the two wins, and the loser either does nothing (wake) or has nothing to cancel (cancel).

13.5.0.3 Idle claim versus idle steal.

A CPU that was claimed just before clearing its own bit finds the donated tasks in its queue and skips the steal.

13.5.0.4 Kill versus creation.

A task is reachable by identifier before it is queued, so a kill can arrive at any time after creation returns its identifier, even before the task has run. The mark is simply seen when the owner first scans the queue.

13.5.0.5 Missed wake-up.

The check that decides whether to wake an idle CPU reads that CPU's state without a lock, so it can race with the target going idle. The target's idle wake interval bounds the resulting delay.

14 Idle State

When a CPU's `cpu_rq` has no runnable tasks, it enters the idle state. The handler selects the idle task, which halts the CPU.

The idle behavior is driven by the scheduling handler, which runs on every wake:

- If the CPU has recently failed to steal, it is in a steal cooldown period. During this time it will not attempt to steal again. Instead it marks itself as available to receive work by setting its bit in the idle mask. This mask allows busy CPUs to proactively hand new tasks to idle CPUs without requiring immediate cross-CPU coordination. The idle CPU does not poll other CPUs directly during cooldown; it simply declares its availability and waits.
- Otherwise it attempts to steal (Section 14.2).

A system with a single CPU never steals.

14.1 Entering Idle

While idle, the CPU executes a low-power halt instruction (e.g., `HLT` on x86 or the platform equivalent). The processor remains halted until the next interrupt.

The wake source is the one-shot timer (if the system allows it), armed at the *idle wake interval* (equal to the priority 9 quantum, 1 ms by default). Another CPU's wake-up, or any other interrupt, also wakes the CPU.

Upon each wake, the handler:

- checks whether any tasks have been added to its `cpu_rq`
- checks whether another CPU has donated tasks to it
- resumes normal scheduling immediately if tasks are available
- if no tasks are available and cooldown has expired, attempts to steal tasks
- if no tasks are obtained, re-arms the wake interval and halts again

This ensures idle CPUs consume minimal power while still remaining responsive.

14.2 Cooldown Expiry

A CPU whose cooldown has expired (or that has never failed) and that still has no runnable tasks will actively attempt to steal tasks from other CPUs. This happens immediately when a CPU runs dry for the first time.

Before the stealing process begins, the targetable bit is cleared in the idle mask to prevent concurrent donation during the steal attempt. If tasks have already arrived in its queue by then, the CPU resumes scheduling without stealing.

If stealing succeeds, the failure count is reset and the CPU resumes normal scheduling.

If stealing fails, the failure count is increased, a new cooldown period begins, and the CPU re-enters idle after setting the targetable bit again.

14.2.1 Cooldown Duration Calculation

Cooldown duration is computed per CPU using exponential backoff. Each CPU tracks a single counter, **failures**: the number of consecutive failed steal attempts since its last successful steal.

Formula 4: Cooldown Duration

<code>cooldown = 0</code>	<code>if failures = 0</code>
<code>cooldown = min(BASE * 2^(failures - 1), CAP)</code>	<code>if failures >= 1</code>

Where:

- **BASE** is the minimum cooldown duration, applied after a single failed attempt. It is one quantum (16 ms) by default.
- **CAP** is the maximum cooldown duration a CPU may reach, regardless of how many consecutive failures it accumulates. It is 1 s by default.
- **failures** increments by 1 on every failed steal attempt, and resets to 0 on the first successful steal.

With the defaults the cap is reached on the seventh consecutive failure.

A CPU under sustained load imbalance in its favor (i.e. one that keeps failing to steal because no other CPU has spare tasks) backs off quickly, reducing wasted steal attempts. A CPU that succeeds resets immediately, so a single successful steal is enough to restore short cooldowns for the next time it runs dry.

14.3 Receiving Tasks While Idle

A CPU that is marked as targetable may receive tasks from another CPU at any time. The idle mask is used when placing new tasks (Section 9.2), including kernel tasks. Balancing moves tasks directly and does not consult the mask.

When a task is placed on an idle CPU:

- The creating CPU first claims the idle CPU by atomically clearing its bit. Only one creator can win a given CPU.
- The task is inserted directly into the idle CPU's `cpu_rq`.
- The idle CPU is woken.

A CPU also withdraws from the mask whenever it selects a task other than the idle task.

This allows work distribution to occur opportunistically without requiring a fully synchronized rebalance operation.

14.4 Kernel Interaction

Kernel tasks may preempt an idle CPU at any time, independent of steal cooldown or idle state rules.

Idle CPUs are the preferred target for new kernel tasks. If any CPU is idle, a kernel task will prefer it over preempting a running task.

When a kernel task targets an idle CPU:

- The CPU is claimed (its bit is cleared in the idle mask) and woken with an interrupt.
- The kernel task begins execution immediately.

If no CPUs are idle, the kernel falls back to its normal preemption behavior as described in [Section 5](#).

15 Platform Requirements

The behavior in this document assumes the platform provides the capabilities below. How they are provided is up to the implementation and the platform port.

- **A clock.** A monotonic, per-CPU microsecond clock (Section 11).
- **A timer.** A per-CPU one-shot timer, with a known minimum interval.
- **Inter-processor interrupts.** A way to wake another CPU, and a way to make a CPU end its running task's quanta. Both can also target the sending CPU.
- **Context switching.** The scheduling handler is entered with interrupts disabled and the interrupted context saved, and it can resume any task's context. A running task can enter the handler on demand (to yield).
- **A low-power halt,** used by the idle task.

15.1 Initialization

Scheduling is brought up in this order:

1. The BSP is CPU 0. Every other processor is numbered in the order it starts, so numbers are contiguous even if a processor fails to start. Processors that have started wait.
2. Once every processor is up, the BSP creates each CPU's idle task and releases the waiting processors.
3. The BSP adopts its boot thread as a task (Section 9.8) and starts preemption.
4. Each released processor enters the scheduling handler with nothing running, and so selects the idle task, or a task already placed on it.

Appendix A: Open Design Considerations

The following are known areas subject to future change:

- **Rebalancing frequency** has been set to 3 s, but it is not yet known how often rebalancing is actually required or how long a pass takes under real load, given that it requires co-operation between CPUs. The value is untuned and subject to change.
- **Blocking task re-insertion** does not currently receive any special treatment. A task is only ever re-scheduled on its next natural turn, in order, once its blocked flag clears. A common pattern is to “boost” unblocked tasks, since they have likely been waiting a long time. This could be achieved by moving the task to the front of its runqueue rather than leaving it in place, but may not prove necessary.
- **Wake latency.** A woken task waits for its next natural turn, so a high-priority task woken behind a long-quanta task can wait up to that task’s full quanta. Only the kernel preempts. If this proves too slow for interrupt-driven work, a bounded form of wake preemption may be needed.
- **Timed waiting.** There is no sleep primitive. This will be changed.
- **Low quanta tasks** (priority 8 and 9, at 2 ms and 1 ms respectively) may have a significant portion of their runtime consumed by context switching. This is not necessarily a problem, as tasks intentionally placed in these tiers are likely doing very little work. With the default priority of 5 providing 16x more runtime than priority 9, very few tasks should ever occupy these tiers unintentionally.
- **Starvation algorithm granularity** could be improved. If a P5 runqueue has 1000 tasks and P2 has 1, P5 tasks will still feel disproportionately slow relative to P2. This is partially by design, but priority boosting could be explored if it becomes a real problem in practice.
- **Load measure.** Balancing counts tasks, not their priority weight, so a CPU with many P9 tasks looks as loaded as one with the same number of P2 tasks.
- **Kill latency** is unbounded. A task marked for destruction is only found when selection scans its queue, which does not happen for a queue the rotation has not reached yet.
- **Kernel task starvation.** A perpetually runnable kernel task starves everything else on its CPU. This is by design, but a cap or a watchdog may be wanted.
- **Hard affinity takeover signals** are not yet defined.
- **Priority Inheritance** may be needed if, for example, a P0 task is relying on something from a P9 task. That P9 task may need to be briefly granted the same priority as the P0 task that needs it.
- **Clock assumptions.** The scheduler only compares a CPU’s clock readings with that same CPU’s earlier readings, but platforms are expected to provide clocks that are consistent across CPUs, because a task can migrate.

Appendix B: Future Considerations

These are recorded here because they are plausible future extensions to the parent/child and destruction model described in Sections 9.5 and 10, not because they are planned work.

- **Wait-for-exit.** A parent blocking until a specific child exits, in the style of POSIX `wait()` / `waitpid()`.
- **Exit status.** A child's termination reason or return value being retained and made observable to its parent, rather than being discarded immediately on destruction.
- **Signal delivery.** A general mechanism for one task to asynchronously notify another, beyond the pending-kill flag used for external destruction (Section 10.5).
- **Better process-hierarchy semantics.** Constructs such as process groups or sessions, should cascading destruction (Section 10.3) alone prove insufficient in practice.

Appendix C: Parameters

Default values for the parameters used in this document. Times are in microseconds.

Parameter	Default	Meaning
Quantum	16000	Base scheduling unit. A priority 5 task runs for one quantum, other levels scale it by powers of two.
Idle wake interval	1000	How often an idle CPU wakes (the priority 9 quantum). Bounds the delay if a wake-up is missed.
Rebalance interval	3000000	Time between periodic balance passes on each CPU.
Minimum imbalance	4	Minimum load difference before a periodic pass moves anything.
Steal batch	4	Maximum tasks an idle CPU steals per attempt.
Steal cooldown base	16000	Steal cooldown after one failure; doubles per further consecutive failure.
Steal cooldown cap	1000000	Upper bound on the steal cooldown.
Default stack size	32 KiB	Stack size when the creator does not specify one.
Kernel task quanta limit	64	Maximum quanta a kernel task may be given per run.

Figure C.1: Parameters

Appendix D: Hard Affinity Tasks

Status: Not Yet Specified

This appendix is a proposal and is not yet part of the specification.

A hard affinity task cannot be moved (Section 4.5), so when its CPU is taken over (Section 5.4) it stops running, possibly forever, without being told. Signaling it at that moment is hard. It may be anywhere in its code, there may be no CPU for it to run on, and there is no signal mechanism yet (Appendix B).

D.1 Takeover Policy

Instead, a hard affinity task states in advance what should happen to it. Each hard affinity task has a *takeover policy*, chosen when it is created, that only applies when its CPU is taken over:

- **STAY**: remain queued on the taken-over CPU until the takeover ends, which may never happen. This is the current behavior, and the default.
- **MOVE**: move to another CPU that is not taken over, keeping hard affinity to the new CPU.
- **DESTROY**: the task is destroyed normally, including its children (Section 10.3).
- **DEMOTE**: lose hard affinity and gain soft affinity to a new CPU, then move like any other soft affinity task.

The taken-over CPU applies the policy itself, on the same passes that redistribute its other tasks. If there is no CPU to move to, MOVE and DEMOTE behave like STAY until there is one.

D.2 Telling the Task

A task that was moved (MOVE or DEMOTE) is told *after* the move, and is not interrupted to be told. It can check an indication that it was moved and which CPU it is on now, so it can rebuild any per-CPU state. The form of that indication is implementation defined. When signals are added, it could be delivered as a signal instead.

If adopted, this replaces the signal described in Section 5.4.

D.3 Open Questions

- Can the policy be changed after creation?
- Do children that inherit their creator's affinity also inherit its policy?
- Where does MOVE go?